Escola Superior de Tecnologia e Gestão	TRABALHO PRÁTICO 2025/2026
Unidade Curricular: Inteligência Artificial	**Trabalho Pratico 01**
Curso: Engenharia Informática	**2º Ano** **2º Semestre**

Labirinto IA — Enunciado do Trabalho Prático (Versão 2)

Fases 1 e 2

Nota: a fase 3 (competição entre grupos) será publicada posteriormente

---

1. Objetivo

Neste trabalho, cada equipa deverá desenvolver um agente autónomo em Python capaz de percorrer um labirinto e recolher prémios. Cada prémio tem um valor inteiro entre 1 e 15, representado no labirinto pelos símbolos hexadecimais 1 a 9 e A a F. O objetivo do agente é maximizar a soma dos valores dos prémios recolhidos durante a partida.

Em cada partida participam dois agentes. Vence o agente que acumular mais pontos quando todos os prémios tiverem sido recolhidos ou quando for atingido o limite máximo de turnos definido para esse labirinto.

O trabalho pretende explorar, entre outras, estratégias de:

- procura não informada;
- procura informada;
- pesquisa local;
- raciocínio estratégico em presença de adversário.

---

2. O labirinto

O ambiente de jogo é um labirinto representado por uma grelha retangular carregada a partir de um ficheiro de texto. Cada célula da grelha contém exatamente um dos seguintes símbolos:

- # — parede, célula bloqueada;
- . — corredor livre;
- 1–9, A–F — prémio com valor entre 1 e 15;
- X — posição inicial do Jogador 1;
- Y — posição inicial do Jogador 2.

Um exemplo de ficheiro de labirinto é fornecido com o enunciado (mais exemplos de labirintos serão fornecidos nos próximos dias). Os labirintos **não podem ser alterados** pelos alunos para efeitos de avaliação. Os alunos poderão, no entanto, criar e usar outros labirintos apenas para testes próprios.

Para cada labirinto existem duas posições iniciais fixas, identificadas por X e Y. Nas fases de avaliação, cada agente jogará pelo menos uma vez a partir de cada uma dessas posições, de forma a reduzir o efeito da posição inicial no resultado. O agente colocado inicialmente na posição X joga sempre primeiro.

Cada célula do labirinto é identificada por um par (linha, coluna), começando em 0 tanto na linha superior como na coluna mais à esquerda.

3. Os agentes

Cada equipa deve entregar um ficheiro Python contendo uma classe que implemente obrigatoriamente os seguintes dois métodos:

```
__init__(maze, prize_positions, agent_position, opponent_position, max_turns)
next_move(maze, prize_positions, agent_position, opponent_position)
```

3.1. Método __init__(...)

Este método é executado uma única vez no início da partida e destina-se à inicialização do agente. Pode ser usado para:

- construir estruturas internas;
- pré-computar distâncias ou caminhos;
- identificar regiões relevantes do mapa;
- preparar qualquer informação que venha a ser usada nas jogadas seguintes.

3.2. Method next_move(...)

Este método é chamado em cada jogada do agente e deverá devolver exatamente uma das seguintes ações:

- Move.UP — mover uma célula para cima;
- Move.DOWN — mover uma célula para baixo;
- Move.RIGHT — mover uma célula para direita;
- Move.LEFT — mover uma célula para esquerda;
- Move.STAY — permanecer na célula atual.

O método *next_move(...)* deve obrigatoriamente devolver um valor do tipo enumerado definido como enumerado no ficheiro *move.py* fornecido com o enunciado.

3.3. Estruturas recebidas pelo agente

Os parâmetros fornecidos aos métodos têm o seguinte significado:

- `maze: tuple[str, …]` - estado atual do labirinto representado por um tuplo com um número de strings igual ao número de linhas do labirinto, correspondendo cada string a uma linha do labirinto com a codificação descrita no ponto 2 começando na parte superior, e cada linha um número de caracteres igual ao número de colunas do labirinto começando do lado esquerdo
- `prize_positions: { (row: int, col: int): prize_value: int }` - dicionário com a localização dos prémios atualmente existentes no labirinto e respetivo valor
  Exemplo: {(1, 3): 4, (2, 7): 10, (5, 2): 4}
- `agent_position: (row: int, col: int)` - Tuplo com a posição atual do próprio agente, no formato (linha, coluna). Na invocação do método *__init__*, este valor corresponde à posição inicial do agente.
- `opponent_position: (row: int, col: int)` - Tuplo com a posição atual do adversário, no formato (linha, coluna). Na invocação do método *__init__*, este valor corresponde à posição inicial do adversário.
- `max_turns` - limite máximo de turnos corresponde ao número máximo de jogadas permitidas para o labirinto, somando as jogadas de ambos os agentes.

As posições são sempre representadas por tuplos (linha, coluna), com indexação iniciada em 0.

Sempre que um prémio é recolhido, a entrada correspondente deixa de existir nesse dicionário. Esta representação permite a existência de vários prémios com o mesmo valor.

O labirinto recebido em cada invocação reflete sempre o estado atual do jogo. Assim, posições onde existiam prémios já recolhidos passam a surgir como corredores livres.

As estruturas recebidas nos métodos do agente devem ser tratadas como estruturas de leitura, não devendo ser modificadas diretamente.

O objeto do agente é criado uma única vez e mantém-se ativo durante toda a partida, pelo que é permitido guardar e atualizar estado interno entre chamadas, através de atributos da classe. Esta possibilidade já estava implícita na versão inicial do documento.

3.4. Restrições

O agente deve ser implementado em Python.

É permitido usar bibliotecas da biblioteca standard de Python e bibliotecas simples de apoio geral, desde que não implementem diretamente algoritmos avançados de procura, planeamento ou exploração de grafos.

Não são permitidas bibliotecas que disponibilizem algoritmos de procura elaborados já prontos a usar, como por exemplo networkx, nem outras bibliotecas equivalentes que permitam contornar o objetivo pedagógico do trabalho.

O uso de threads, multiprocessing ou qualquer outro mecanismo de execução paralela ou concorrente não é permitido. O agente deve executar toda a sua lógica de forma síncrona, dentro do tempo de execução dos métodos __init__(...) e next_move(...)

O objetivo é que os algoritmos de procura e decisão sejam implementados pelos alunos.

---

4. Mecânica dos turnos

Nas **fases 1 e 2**, o jogo decorre por turnos alternados simples.

Em cada jogada, apenas um agente atua. Depois de concluída essa jogada, passa a vez ao outro agente. Isto é, os dois agentes jogam alternadamente.

O agente que começa a partida é sempre o agente colocado na posição X.

Em cada jogada, a sequência de execução é a seguinte:

1. o agente ativo recebe o estado atual do jogo;
2. o agente devolve uma ação;
3. a ação é validada;
4. a posição do agente é atualizada;
5. se o agente entrar numa célula com prémio, esse prémio é recolhido e os pontos correspondentes são adicionados ao seu resultado;
6. a vez passa ao adversário.

---

5. Movimentos, validação e recolha de prémios

Se um agente tentar executar um movimento que o leve para uma parede, o movimento é considerado inválido e o agente permanece na sua posição atual.

Se o método *next_move(...)* devolver um valor diferente de Move.UP, Move.DOWN, Move.RIGHT, Move.LEFT ou Move.STAY, ou se ocorrer uma exceção durante a sua execução, considera-se que o agente executa a ação Move.STAY nessa jogada. Da mesma forma, se o método não devolver nada

explicitamente (*None*), também conta como inválido e considera-se que o agente executa a ação Move.STAY.

Dois agentes podem ocupar a mesma célula em simultâneo. Esta situação é tratada como um cruzamento, não existindo qualquer penalização para nenhum deles.

Um prémio é recolhido no instante em que o agente ativo **entra** numa célula que contenha esse prémio. Após a recolha:

- o prémio desaparece do labirinto;
- a entrada correspondente é removida do parâmetro *prize_positions*;
- o valor do prémio é adicionado à pontuação do agente.

Como apenas um agente se move em cada turno, nunca existe recolha simultânea do mesmo prémio pelos dois agentes. O primeiro agente a entrar nessa célula recolhe o prémio.

---

6. Fim do jogo

Uma partida termina quando se verificar uma das seguintes condições:

- todos os prémios tiverem sido recolhidos
- ou for atingido o limite máximo de turnos definido para o labirinto.

O limite máximo de turnos corresponde ao número total de jogadas da partida, somando as jogadas de ambos os agentes.

No final da partida:

- vence o agente com maior soma de pontos;
- se ambos os agentes tiverem a mesma pontuação, o resultado é empate.

---

7. Fases de avaliação

As fases 1 e 2 integram a avaliação do trabalho. Em cada fase, cada agente realizará jogos suficientes para jogar pelo menos uma vez a partir de cada posição inicial possível.

A fase 1 incide na correta implementação do agente e na sua capacidade de interagir corretamente com o motor de jogo, sendo tido em conta a soma dos valores dos prémios recolhidos dentro do limite de jogadas definidas.

A fase 2 incide no desempenho do agente face aos adversários de referência, considerando os resultados obtidos nas partidas realizadas.

Em cada partida, o resultado é determinado pela soma dos valores dos prémios recolhidos até ao final do jogo. Para efeitos de avaliação, considera-se o desempenho obtido nas partidas realizadas contra os adversários de referência fornecidos.

Para testar as fases 1 e 2 serão fornecidos labirintos de exemplo.

Fase 1 — Validação funcional

Cada agente joga contra um adversário *Dummy*, que permanece sempre parado. Esta fase decorre num labirinto de pequena dimensão. Os turnos são alternados de forma simples. O objetivo desta fase é:

- validar o correto funcionamento do agente;
- confirmar que a interface foi implementada corretamente;
- obter uma primeira referência de desempenho.

Cada agente realizará partidas começando pelo menos uma vez em X e uma vez em Y.

Fase 2 — Comparação com agentes de referência

Cada agente joga contra adversários de referência fornecidos pelo docente, em labirintos de dimensão média. Os turnos continuam a ser alternados de forma simples. O objetivo desta fase é avaliar a qualidade da estratégia desenvolvida antes de qualquer fase competitiva mais avançada.

Os dois adversários de referência, por ordem crescente de dificuldade, são:

- *Dummy* — permanece sempre parado;
- Aleatório — escolhe aleatoriamente um movimento válido para uma célula livre;

Adversários de referência adicionais poderão ser fornecidos.

Os agentes que implementam os adversários Dummy e Aleatório são fornecidos com o presente enunciado.

---

8. Elementos a entregar

Cada grupo deverá submeter no Moodle um ficheiro zip com os seguintes elementos:

- Ficheiro com código python **groupNN.py** em que ***NN*** corresponde ao número do grupo representado com 2 dígitos, conforme escolhido no Moodle. Por exemplo: group05.py ou group23.py
- O nome da classe que implementa o agente deverá ser **Group*NN*** em que ***NN*** corresponde ao número do grupo representado com 2 dígitos, conforme escolhido no Moodle. Por exemplo: `class Group05:` ou `class Group23:`
- Ficheiro PDF com a explicação das técnicas utilizadas na estratégia do agente de acordo com o template fornecido com o enunciado

---

9. Código fonte

9.1. Estrutura de Ficheiros

O projeto é fornecido num ficheiro comprimido que deve ser descomprimido numa pasta.

A estrutura do projeto é a seguinte:

```
project/
├── run.py
├── move.py
├── ...
├── mazes/
│   └── maze01.txt
├── viewer/
│   └── ...
└── agents/
    ├── __init__.py
    ├── dummy_agent.py
    ├── random_agent.py
    ├── simpleHC_agent.py
    ├── ...
    └── group99.py
```

onde:

- `move.py` — define o enumerado `Move` com as cinco ações possíveis: `Move.UP, Move.DOWN, Move.RIGHT, Move.LEFT, Move.STAY`.
- `run.py` — script de entrada utilizado para executar um jogo a partir da linha de comandos (ver secção 9.2).
- `mazes/` — pasta com os ficheiros de labirinto fornecidos.
- `agents/` — pasta com todos os ficheiros de agentes. O ficheiro `__init__.py` deve estar presente (pode estar vazio). Os agentes de referência (`dummy_agent.py`, `random_agent.py`, etc.) e o template dos alunos (`group99.py`) devem todos ser colocados nesta pasta.

9.2. Execução de um Jogo

Os jogos são executados a partir da linha de comandos usando o script `run.py`, a partir da pasta raiz do projeto. A sintaxe é:

```
python run.py <ficheiro_labirinto> <max_turnos> <agente1> <agente2> [options]
```

em que:

- `<ficheiro_labirinto>` — caminho para o ficheiro de labirinto (p.ex. `mazes/maze01.txt`).
- `<max_turnos>` — número máximo total de turnos do jogo (inteiro).
- `<agente1>` — nome do Agente 1. Usar `dummy` ou `random` para os agentes de referência, ou o nome do ficheiro do grupo sem extensão (p.ex. `group05`) para um agente de aluno.
- `<agente2>` — nome do Agente 2, seguindo a mesma convenção de `<agente1>`.
- `[options]` — opções de visualização (ver help do programa, executando `python run.py`)

Exemplos:

- Executar o agente contra o adversário de referência simpleHC, com limite de 100 turnos:
  `python run.py mazes/maze01.txt 100 group05 simpleHC -v text`
- Executar o agente contra o adversário de referência random, com limite de 200 turnos:
  `python run.py mazes/maze01.txt 200 group05 random -v color`
- Executar dois agentes de alunos entre si, com limite de 200 turnos:
  `python run.py mazes/maze01.txt 200 group05 group12`

Note-se que `<agente1>` é sempre colocado na posição inicial X (joga primeiro) e `<agente2>` na posição Y. Para avaliar o agente de forma equitativa, deve executar-se o jogo com as posições trocadas, invertendo a ordem dos argumentos.

---

10. Fase 3

A fase 3 inclui um campeonato em que todos os agentes jogam entre si em labirintos com diferentes configurações (serão fornecidos exemplos de labirintos).

Cada par de grupos disputa duas partidas no mesmo labirinto, trocando as posições iniciais e quem começa, a fim de garantir equidade (nenhuma equipa é favorecida pela posição de partida).

Para cada grupo e cada labirinto, é registada a soma dos tempos de todas as jogadas do grupo (incluindo o tempo de execução do __init__(...) e next_move(...)) e a soma do valor dos prémios perdidos (isto é, recolhidos pelos adversários), em todos os jogos disputados contra todos os outros grupos.

A ordenação dos grupos será realizada recorrendo ao ranking por camadas de Pareto considerando os seguintes dois objetivos: minimizar o tempo acumulado das jogadas e minimizar o valor acumulado dos prémios perdidos. O ranking por camadas de Pareto funciona da seguinte forma: atribui a posição 1 a todos os grupos da fronteira de Pareto. A seguir, remove os grupos da fronteira de

Pareto (camada 1), recalcula a fronteira de Pareto sobre os restantes (camada 2), atribuindo-lhes a posição 2. O procedimento repete-se sucessivamente até completar todos os grupos.

A posição final de cada grupo será dada pela soma das posições conseguidas em cada labirinto do campeonato (o grupo com menor soma fica em primeiro lugar).

.

11. Avaliação

Os trabalhos práticos serão avaliados tendo em conta os seguintes critérios:

- Requisitos mínimos obrigatórios (para que o trabalho prático tenha avaliação positiva):
  - Fase 1: o agente deve conseguir recolher todos os prémios contra o jogador Dummy, para os seguintes labirintos: fase1_maze1.txt, fase1_maze2.txt, fase1_maze3.txt;
  - Fase 2: o agente deve demonstrar a capacidade de competir contra agentes fornecidos (recolhendo pelo menos um dos prémios existentes no início do jogo)
  - Apresentação na aula prática do dia 09/04/2026
- Técnica(s) utilizada(s) no desenvolvimento do agente
- Partidas ganhas contra os agentes fornecidos na fase 2, considerando os objetivos de minimizar o tempo acumulado das jogadas e minimizar o valor acumulado dos prémios perdidos (para o ranking será utilizado o ranking de camadas de Pareto).
- Posição conseguida no campeonato descrito na fase 3.